Secure Software Development Standard (SSDS)

Version: 2026.04
Status: Internal Use Only
Last Reviewed: Date
Approved By: Person

1. Introduction and Purpose

This standard outlines the mandatory security requirements for software development and source code management. As a scientific research organisation, we encourage open collaboration; however, we must balance this with the protection of intellectual property and national security interests. This document provides a framework to ensure code integrity, prevent credential leakage, and secure "Critical Infrastructure" projects.

2. Classification-Based Repository Governance

Project classification dictates the allowed hosting environment. Failure to adhere to these boundaries constitutes a serious security breach.

Project Classification	Definition	Permitted Hosting
Open Research	General scientific inquiry with no commercial or security sensitivities.	GitHub (Public or Private)
Standard Internal	Proprietary operational tools or internal-use research.	Internal GitLab / Enterprise GitHub
Strategic/Critical	Projects designated as Critical Infrastructure or National Security interest.	Air-gapped / Secure Internal Repo

2.1 Critical Infrastructure Prohibitions

Projects identified as **Strategic/Critical** must never be hosted on external SaaS platforms like GitHub. These projects require:

- Storage on physically isolated or high-assurance internal networks.
- Mandatory use of "Security Agent" sign-off for all internal pull requests.
- Strict prohibition of external contributors without manual security vetting.

3. Secure Coding Requirements

All code, regardless of classification, must adhere to these foundational security principles during the development lifecycle.

3.1 Secret and Credential Management

- **Zero-Credential Policy:** No API keys, passwords, connection strings, or cryptographic keys shall be hardcoded in any source file.
- **Scanning:** All pushes to GitHub must undergo pre-commit or server-side scanning for secrets using approved tools (e.g., Snyk, Trufflehog).
- **Injection:** Use environment variables or approved Secret Management platforms (e.g., Vertex AI Secret Manager).

3.2 Supply Chain & Dependencies

- **Vulnerability Scanning:** Developers must use tools like Snyk to scan third-party libraries for known vulnerabilities (CVEs) before integration.
- **Pinned Versions:** Avoid using "latest" tags; use specific version numbers or hashes to ensure build reproducibility and security.

3.3 Artificial Intelligence & Model Security

In alignment with modern ISM controls:

- **Non-Executable Models:** AI models must be stored in non-executable formats (e.g., Safetensors) to prevent arbitrary code execution.
- **Data Isolation:** Organisational data used for fine-tuning must not be used to train external foundation models without explicit consent.

4. Automated Enforcement Workflow

The organization utilizes an automated pipeline to maintain these standards:

1. **Developer Action:** Code is developed in VS Code leveraging Gemini Code Assist for secure pattern suggestions.

2. **Commit/Push:** Developer initiates a push via Gemini CLI or standard Git commands.
3. **Security Gate (MCP Servers):** MCP Servers intercept the push and initiate a Snyk scan for vulnerabilities and secrets.
4. **Remediation:** If issues are found, the system proposes fixes; if the project is "Critical Infrastructure," the push is immediately blocked and escalated to the TDA.
5. **Review:** A Code Review Agent provides an automated summary of the security posture before human approval.

5. Compliance and Escalation

Deviations from this standard must be documented and approved by the Technical Design Authority (TDA).

- **In-Scope:** Standard scientific and enterprise applications.
- **Out-of-Scope:** Requests falling outside the seven SME subagent domains must follow the standard IM&T escalation path.

Related Documentation:

- 🗋 File [CSIRO Security & Privacy Standards 2026]
- 🗋 File [Data Handling Policy for Research]